

Parallel Programming Patterns

Overview and Concepts



Reusing this material



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

http://creativecommons.org/licenses/by-nc-sa/4.0/deed.en_US

This means you are free to copy and redistribute the material and adapt and build on the material under the following terms: You must give appropriate credit, provide a link to the license and indicate if changes were made. If you adapt or build on the material you must distribute your work under the same license as the original.

Acknowledge EPCC as follows: “© EPCC, The University of Edinburgh, www.epcc.ed.ac.uk”

Note that this presentation contains images owned by others. Please seek their permission before reusing these images.

Outline

- Why parallel programming?
- Decomposition
 - Geometric decomposition
 - Task farm
 - Pipeline
 - Loop parallelism
- Performance metrics and scaling
 - Amdahl's law
 - Gustafson's law

Why use parallel programming?

It is harder than serial so why bother?

Why?

- Parallel programming is more difficult than its sequential counterpart
- However we are reaching limitations in uniprocessor design
 - Physical limitations to size and speed of a single chip
 - Developing new processor technology is very expensive
 - Some fundamental limits such as speed of light and size of atoms
- Parallelism is not a silver bullet
 - There are many additional considerations
 - Careful thought is required to take advantage of parallel machines

Performance

- A key aim is to solve problems faster
 - To improve the time to solution
 - Enable new scientific problems to be solved
- To exploit parallel computers, we need to split the program up between different processors
- Ideally, would like program to run P times faster on P processors
 - Not all parts of program can be successfully split up
 - Splitting the program up may introduce additional overheads such as communication

Parallel tasks

- How we split a problem up in parallel is critical
 1. Limit communication (especially the number of messages)
 2. Balance the load so all processors are equally busy
- Tightly coupled problems require lots of interaction between their parallel tasks
- Embarrassingly parallel problems require very little (or no) interaction between their parallel tasks
 - E.g. the image sharpening exercise
- In reality most problems sit somewhere between two extremes

Decomposition

How do we split problems up to solve efficiently in parallel?

Decomposition

- One of the most challenging, but also most important, decisions is how to split the problem up
- How you do this depends upon a number of factors
 - The nature of the problem
 - The amount of communication required
 - Support from implementation technologies
- We are going to look at some frequently used decompositions
 - will be illustrated by later Fractal and CFD practical examples

Geometric decomposition

- Take advantage of the geometric properties of a problem

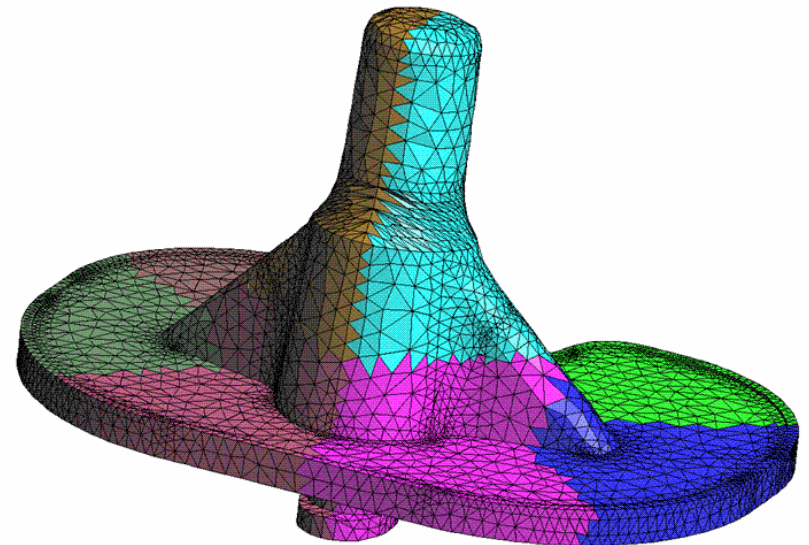
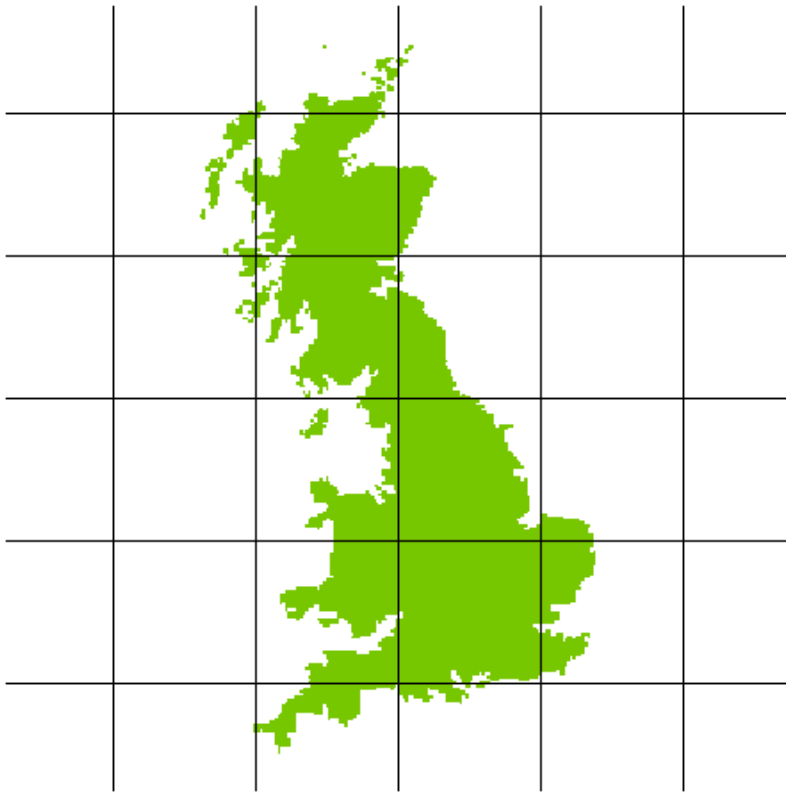
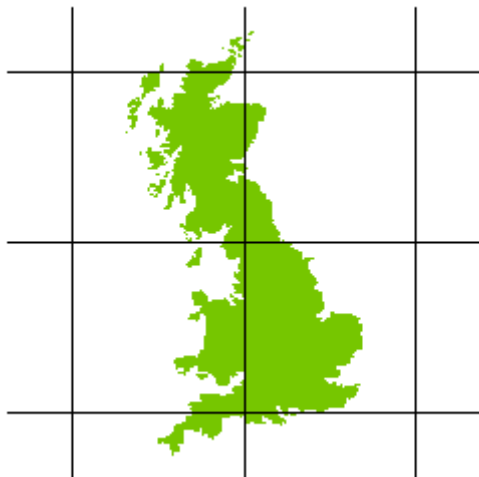


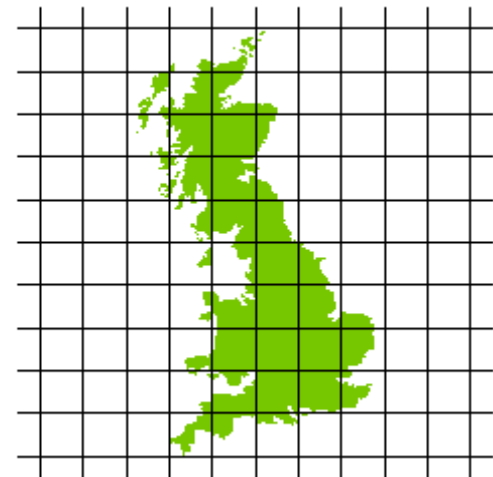
Image from ITWM: <http://www.itwm.fraunhofer.de/en/departments/flow-and-material-simulation/mechanics-of-materials/domain-decomposition-and-parallel-mesh-generation.html>

Geometric decomposition

- Splitting the problem up does have an associated cost
 - Namely communication between processors
 - Need to carefully consider granularity
 - Aim to minimise communication and maximise computation



- Granularity
 - size of chunks of work



- Chunks too large
 - too little parallelism

- Chunks too small
 - communications rule

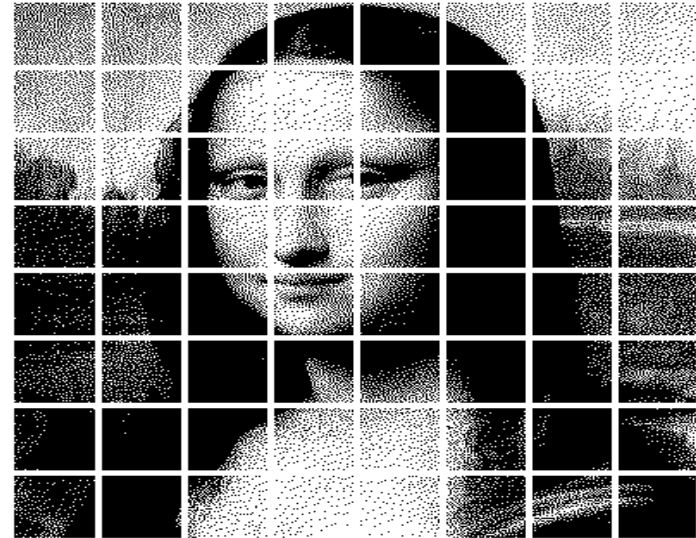
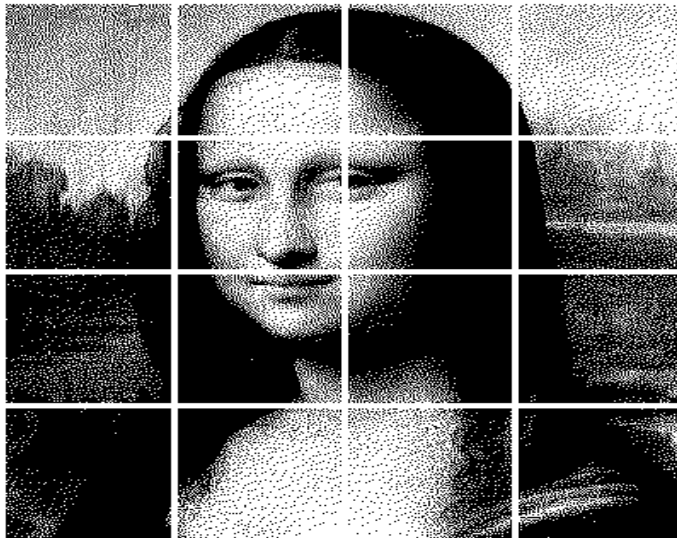
Halo swapping

- Swap data in bulk at pre-defined intervals
- Often only need information on the boundaries
- Many small messages result in far greater overhead



Load imbalance

- Execution time determined by slowest processor
 - each processor should have (roughly) the same amount of work, i.e. they should be load balanced

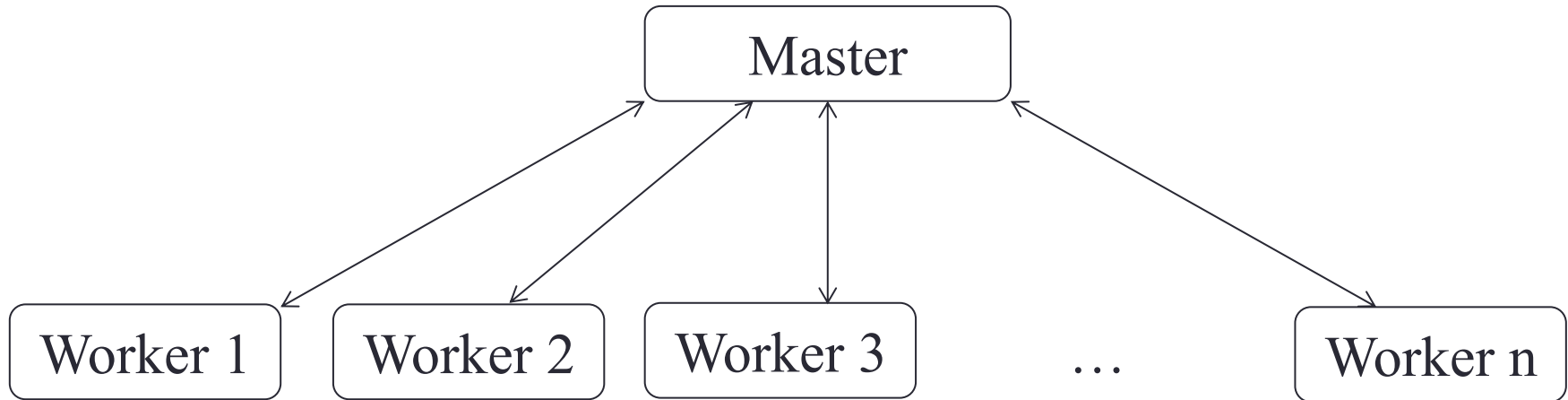


- Assign multiple partitions per processor
 - see Fractal example
 - Additional techniques such as work stealing available

Task farm (master worker)

Fractal

- Split the problem up into distinct, independent, tasks



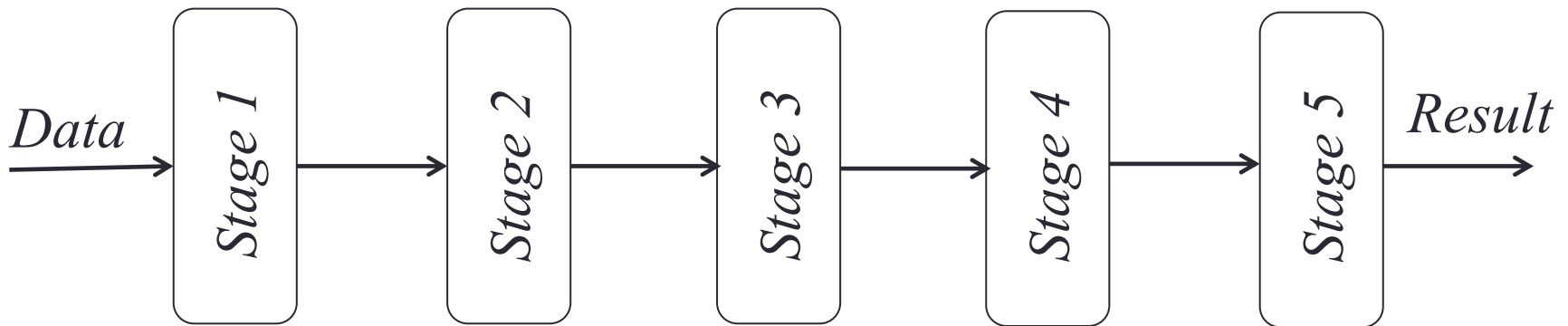
- Master process sends task to a worker
- Worker process sends results back to the master
- The number of tasks is often much greater than the number of workers and tasks get allocated to idle workers

Task farm considerations

- Communication is between the master and the workers
 - Communication between the workers can complicate things
- The master process can become a bottleneck
 - Workers are idle waiting for the master to send them a task or acknowledge receipt of results
 - Potential solution: implement work stealing
- Resilience – what happens if a worker stops responding?
 - Master could maintain a list of tasks and redistribute that work's work

Pipelines

- A problem involves operating on many pieces of data in turn. The overall calculation can be viewed as data flowing through a sequence of stages and being operated on at each stage.



- Each stage runs on a processor, each processor communicates with the processor holding the next stage
- One way flow of data

Example: pipeline with 4 processors



- Each processor (one per colour) is responsible for a different task or stage of the pipeline
- Each processor acts on data (numbered) as they move through the pipeline

Examples of pipelines

- CPU architectures
 - Fetch, decode, execute, write back
 - Intel Pentium 4 had a 20 stage pipeline
- Unix shell
 - i.e. `cat datafile | grep "energy" | awk '{print $2, $3}'`
- Graphics/GPU pipeline
- *A generalisation of pipeline (a workflow, or dataflow) is becoming more and more relevant to large, distributed scientific workflows*
- *Can combine the pipeline with other decompositions*

Loop parallelism

- Serial programs can often be dominated by computationally intensive loops.
- Can be applied incrementally, in small steps based upon a working code
 - This makes the decomposition very useful
 - Often large restructuring of the code is not required
 - e.g. compare different parallelisations for later CFD exercise
- Tends to work best with small scale parallelism
 - Not suited to all architectures
 - Not suited to all loops
- If the runtime is not dominated by loops, or some loops can not be parallelised then these factors can dominate (Amdahl's law.)

Example of loop parallelism:

```
int main(int argc, char *argv[])
{
    const int N = 1000000;
    int i, a[N];

    #pragma omp parallel for
    for (i=0; i < N; i++)
        a[i] = 2 * a[i];

    return 0;
}
```

- If we ignore all parallelisation directives then should just run in serial
- Technologies have lots of additional support for tuning this

Performance metrics and scaling

How is my parallel code performing and scaling?

Performance metrics

- Measure the execution time T
 - how do we quantify performance improvements?

- Speed up
 - typically $S(N,P) < P$

$$S(N,P) = \frac{T(N,1)}{T(N,P)}$$

- Parallel efficiency
 - typically $E(N,P) < 1$

$$E(N,P) = \frac{S(N,P)}{P} = \frac{T(N,1)}{P T(N,P)}$$

- Serial efficiency
 - typically $E(N) \leq 1$

$$E(N) = \frac{T_{best}(N)}{T(N,1)}$$

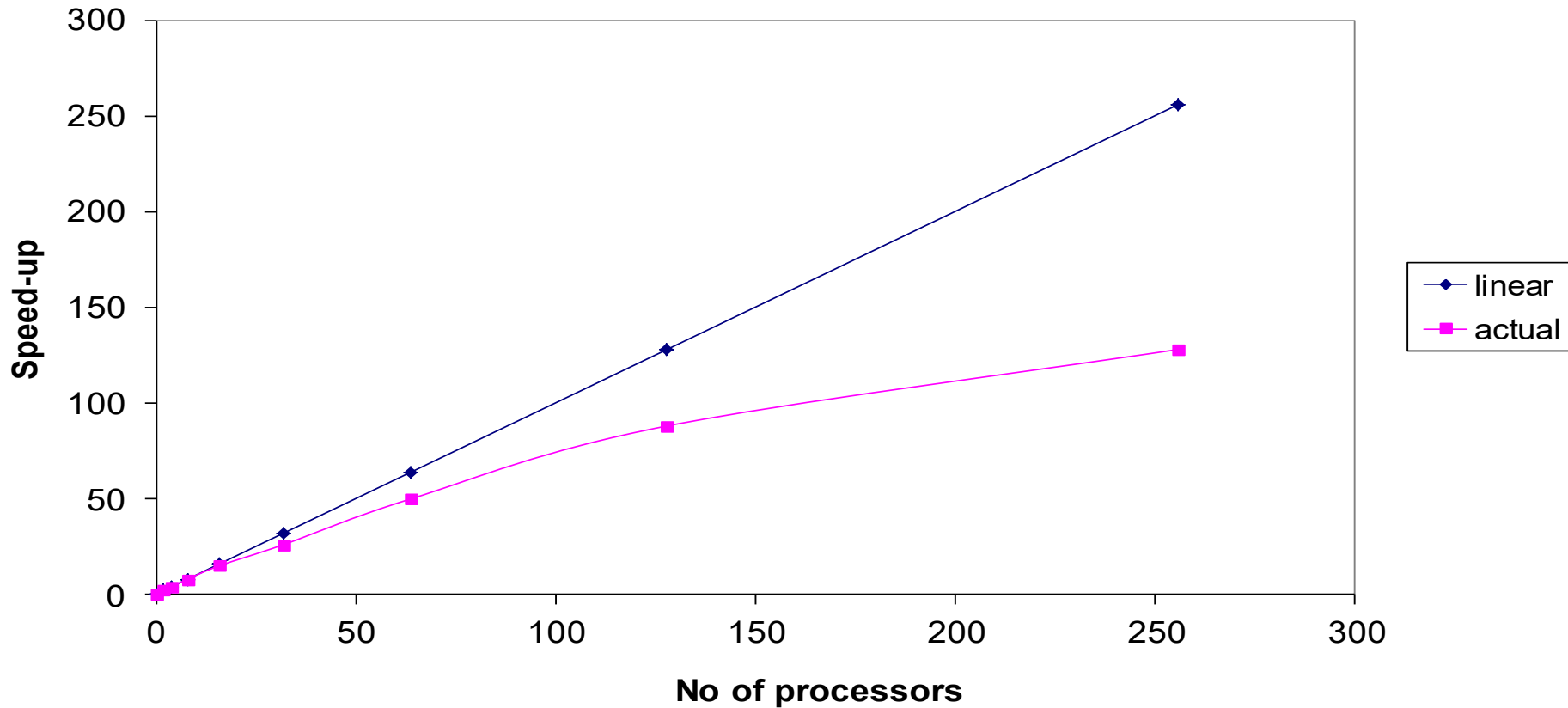
Where N is the size of the problem and P the number of processors

Scaling

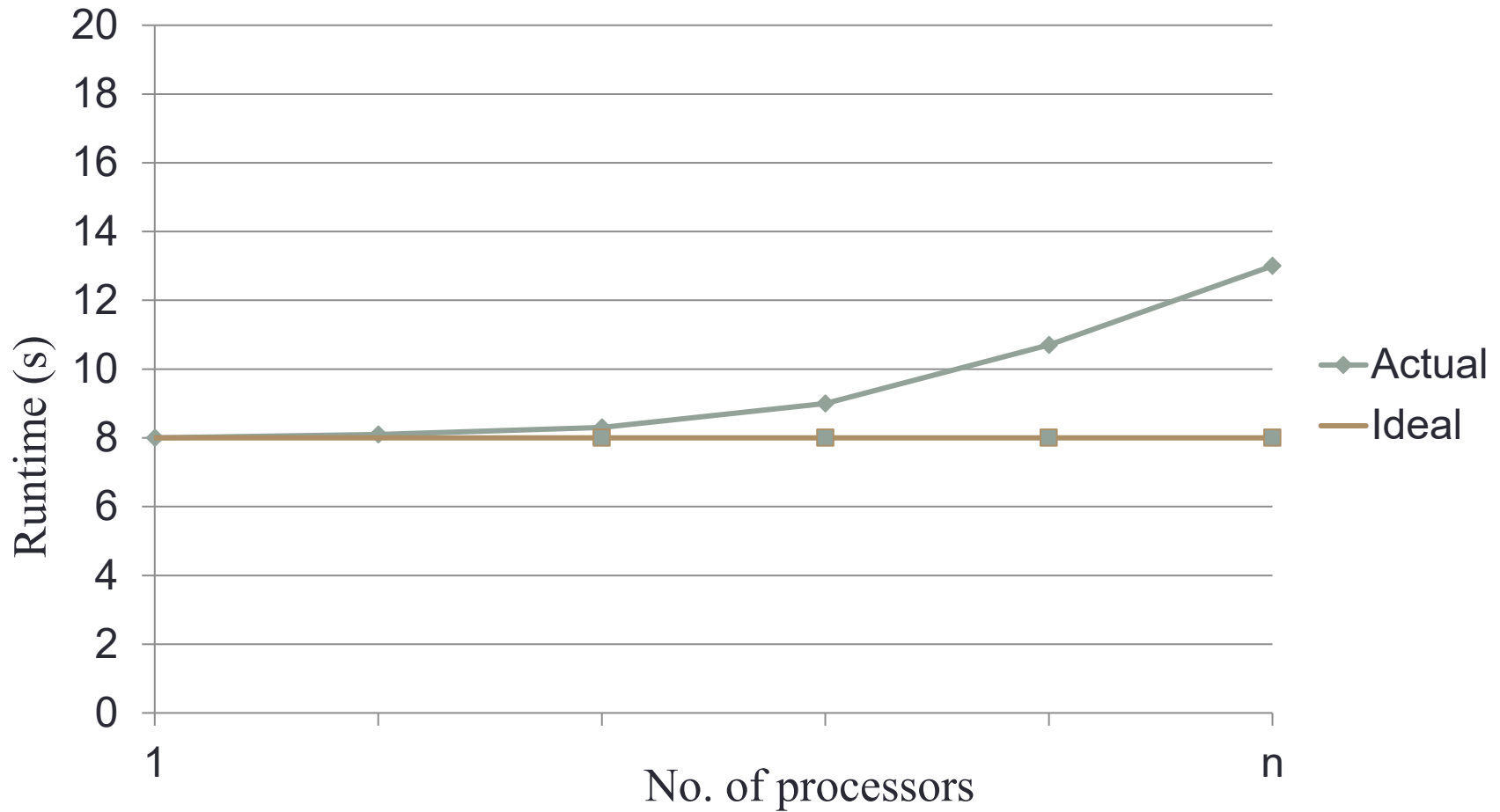
- *Scaling* is how the performance of a parallel application changes as the number of processors is increased
- There are two different types of scaling:
 - *Strong Scaling* – total problem size stays the same as the number of processors increases
 - *Weak Scaling* – the problem size increases at the same rate as the number of processors, keeping the amount of work per processor the same
- Strong scaling is generally more useful and more difficult to achieve than weak scaling

Strong scaling

Speed-up vs No of processors



Weak scaling



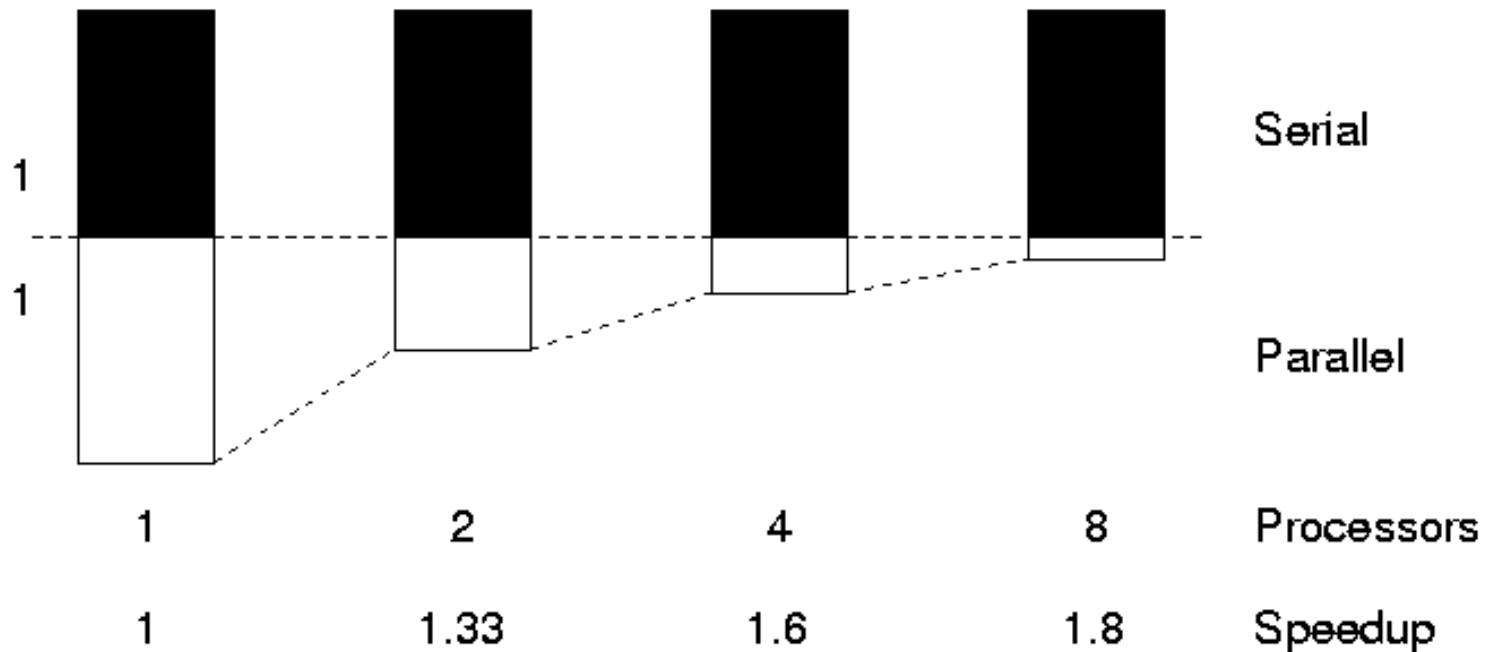
The serial fraction

An inherent limit to speed up when we parallelise problems

The serial section of code

“The performance improvement to be gained by parallelisation is limited by the proportion of the code which is serial”

Gene Amdahl, 1967



- A typical program has two categories of components
 - Inherently sequential sections: can't be run in parallel
 - Potentially parallel sections
- Assume fraction α is serial and parallel part is 100% efficient:

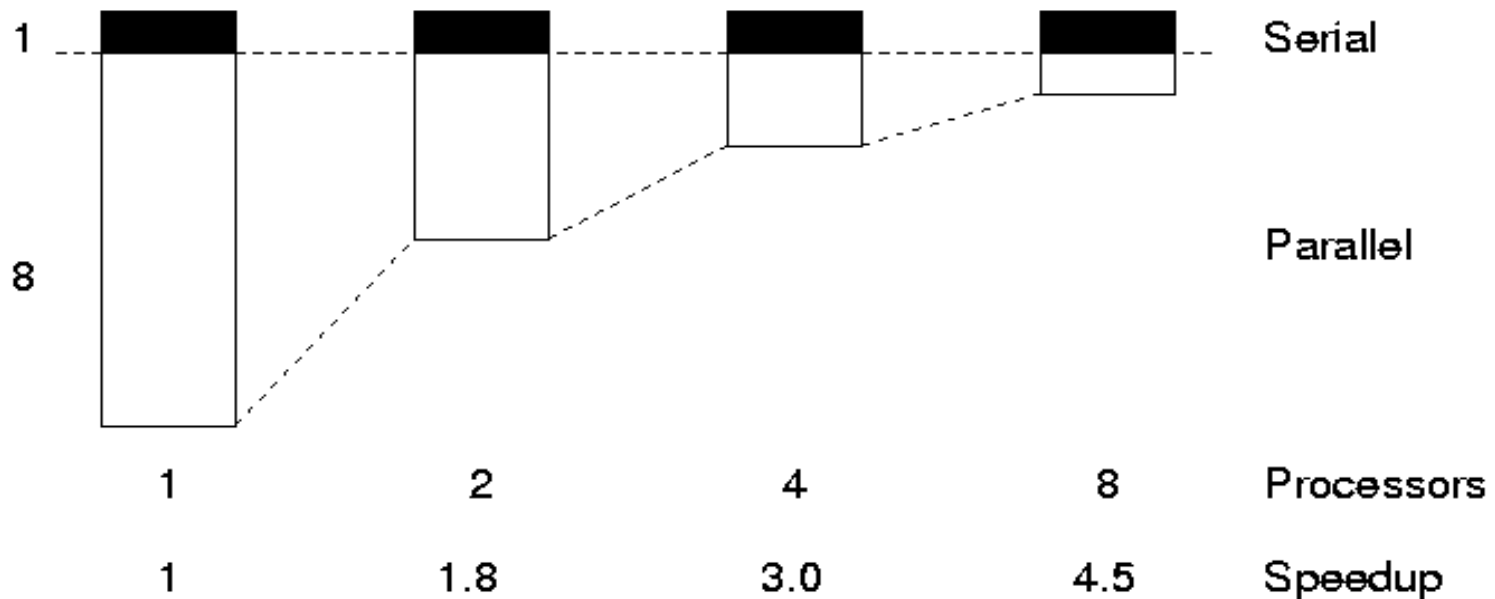
- Parallel runtime
$$T(N, P) = \alpha T(N, 1) + \frac{(1 - \alpha)T(N, 1)}{P}$$

- Parallel speedup
$$S(N, P) = \frac{T(N, 1)}{T(N, P)} = \frac{P}{\alpha P + (1 - \alpha)}$$

- We are fundamentally limited by the serial fraction
 - For $\alpha = 0$, $S = P$ as expected (i.e. *efficiency* = 100%)
 - Otherwise, speedup limited by $1/\alpha$ for any P
 - For $\alpha = 0.1$; $1/0.1 = 10$ therefore 10 times maximum speed up
 - For $\alpha = 0.1$; $S(N, 16) = 6.4$, $S(N, 1024) = 9.9$

Gustafson's Law

- We need larger problems for larger numbers of CPUs



- Whilst we are still limited by the serial fraction, it becomes less important

Utilising Large Parallel Machines

- Assume parallel part is proportional to N

- and that serial fraction α is independent of N

- time
$$T(N, P) = T_{serial}(N, P) + T_{parallel}(N, P)$$
$$= \alpha T(1, 1) + \frac{(1 - \alpha)NT(1, 1)}{P}$$

- speedup
$$S(N, P) = \frac{T(N, 1)}{T(N, P)} = \frac{\alpha + (1 - \alpha)N}{\alpha + (1 - \alpha)\frac{N}{P}}$$

- Scale problem size with CPUs, i.e. set $N = P$ (weak scaling)

- speedup
$$S(P, P) = \alpha + (1 - \alpha)P$$

- efficiency
$$E(P, P) = \alpha/P + (1 - \alpha)$$

Gustafson's Law

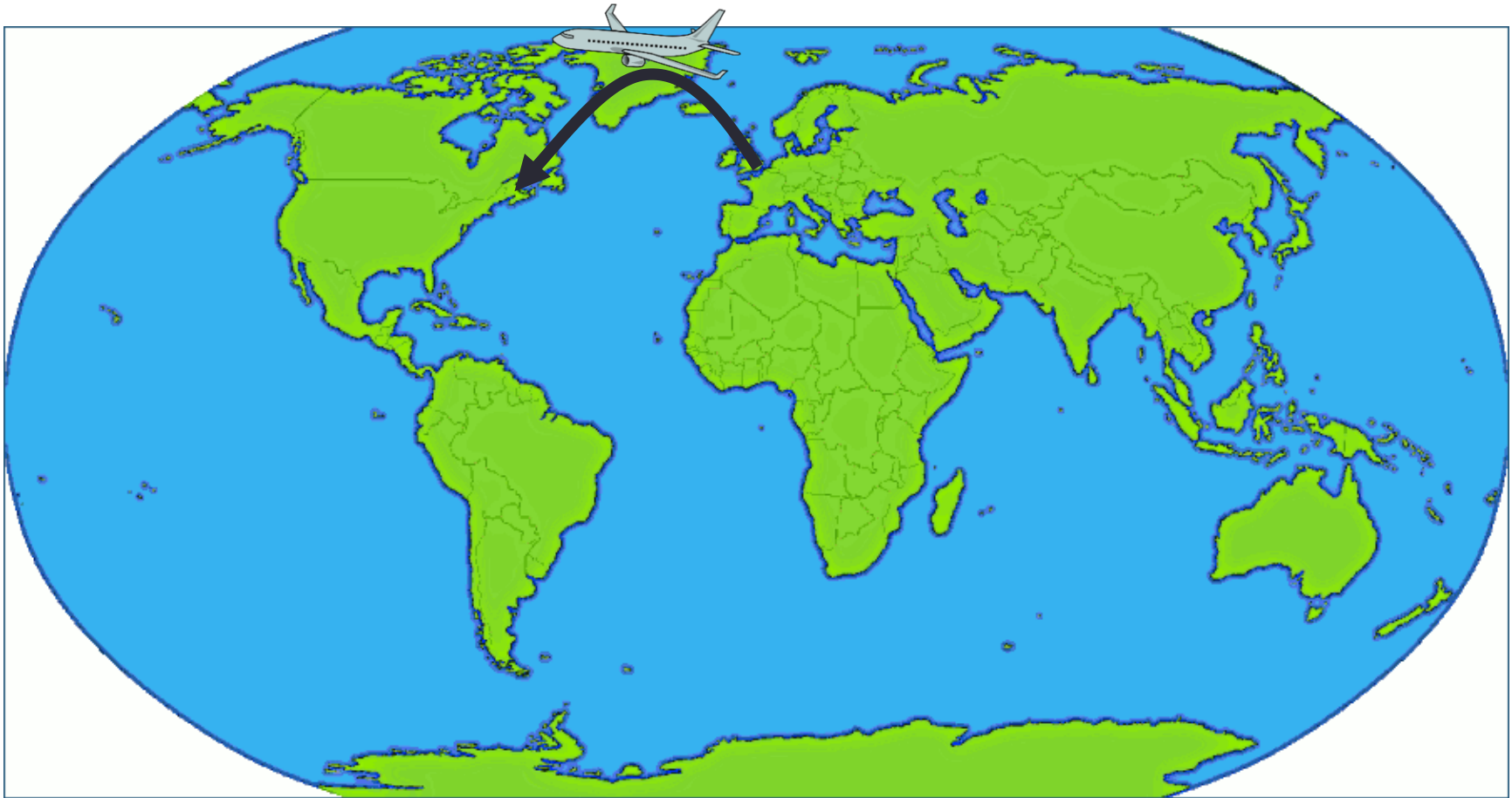
CFD

- If you increase the amount of work done by each parallel task then the serial component will not dominate
 - Increase the problem size to maintain scaling
 - Can do this by adding extra complexity or increasing the overall problem size
- Assume 10% serial fraction for initial problem size:

Number of processors	Strong scaling (Amdahl's law)	Weak scaling (Gustafson's law)
16	6.4	14.5
1024	9.9	922

Due to the scaling of N , the serial fraction effectively becomes α/P

Analogy: Flying London to New York



Buckingham Palace to Empire State

- By airplane
 - Distance: 5600 km; speed: 600 mph
 - Flight time: 8 hours
- But.....
 - 2 hours to check in at the airport in London
 - 2 hours to get through immigration & collect bag in NY
 - Fixed overhead of 4 hours; total journey time: $4 + 8 = 12$ hours
- Triple the flight speed with Concorde to 1800 mph
 - Flight time: 2 hours 40 mins
 - But still need to spend 4 hours in airports
 - Total journey time = 2hrs 40 mins + 4 hours = 6 hrs 40 mins
 - Speedup of 1.8 not 3.0
- Amdahl's law! $\alpha = 4/12 = 0.33$; max speedup = 3 (i.e. 4 hours)

Flying London to Sydney



Buckingham Palace to Sydney Opera

- By airplane
 - Distance: 14400 miles; speed: 600 mph; flight time; 24 hours
 - Serial overhead **stays the same**
 - total time: $4 + 24 = 28$ hours
- Triple the flight speed
 - Total time = 4 hours + 8 hours = 12 hours
 - Speedup = 2.3 (as opposed to 1.8 for New York)
- Gustafson's law!
 - Bigger problems scale better
 - Increase **both** distance (i.e. N) **and** max speed (i.e. P) by three
 - Maintain same balance: 4 “serial” + 8 “parallel”

Load imbalance

Keeping processors equally busy

Load Imbalance

- These laws all assumed all processors are equally busy
 - But what happens if some run out of work?

- Specific case

- Four people pack boxes with cans of soup: 1 minute per box

Person	Anna	Paul	David	Helen	Total
# boxes	6	1	3	2	12

- Takes 6 minutes as everyone is waiting for Anna to finish!
 - If we gave everyone same number of boxes, would take 3 minutes
- Scalability isn't everything
 - Make the best use of the processors at hand before increasing the number of processors

Quantifying Load Imbalance

- Define Load Imbalance Factor

$$LIF = \text{maximum load} / \text{average load}$$

- For perfectly balanced problems $LIF = 1.0$, as expected
 - In general, $LIF > 1.0$
 - LIF tells you how much faster your calculation could be with balanced load
- Box packing
 - $LIF = 6/3 = 2$
 - Initial time = 6 minutes
 - Best time = 6 minutes / 2 = 3 minutes

Summary

Summary

- There are many considerations when parallelising code
- A variety of patterns exist that can provide well known approaches to parallelising a serial problem
 - You will see examples of some of these during the practical sessions
- Scaling is important, as the more a code scales the larger a machine it can take advantage of
 - can consider weak and strong scaling
 - in practice, overheads limit the scalability of real parallel programs
 - Amdahl's law models these in terms of serial and parallel fractions
 - larger problems generally scale better: Gustafson's law
- Load balance is also a crucial factor
- Metrics exist to give you an indication of how well your code performs and scales